Syntaxanalyse: LR-Parser (LR(0), LALR)

BC George (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Wiederholung

Top-Down-Analyse

- Baumaufbau von oben nach unten
- eine Möglichkeit: recursive-descent parser
- alternativ: tabellengesteuerter Parser
- *First*- und *Follow*-Mengen bestimmen Wahl der Ableitungen
- nicht mehr rekursiv, sondern mit PDA

Motivation

LL ist nicht alles

Die Menge der LL-Sprachen ist eine echte Teilmenge der deterministisch kontextfreien Sprachen.

Bei LL-Sprachen muss man nach den ersten k Eingabezeichen entscheiden, welche Ableitung ganz oben im Baum als erste durchgeführt wird, also eine, die im Baum ganz weit weg ist von den Terminalen, die die Entscheidung bestimmen. Das ist nicht bei allen deterministisch parsebaren Grammatiken möglich und erschwert die Fehlerbehandlung.

Von unten nach oben

Bei der Bottom-Up-Analyse wird der Parse Tree wird von unten nach oben aufgebaut, von links nach rechts. Dabei entsteht eine *Rechtsableitung*.

Def.: Bei einer kontextfreien Grammatik G ist die *Rechtsableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn das am weitesten rechts stehende Nichtterminal in α abgeleitet wird. Man schreibt $\alpha \overset{*}{\Rightarrow}_r \beta$.

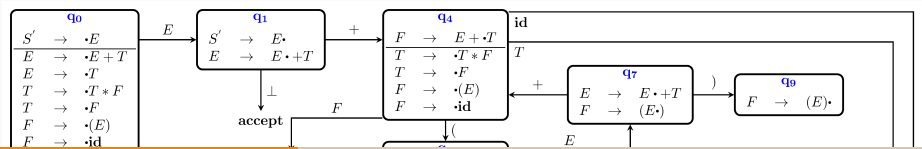
Mit Hilfe der Produktionen und der Vorschautoken werden die Ableitungen "rückwärts" angewandt und "Reduktionen" genannt.

Versuchen wir es einmal

Kann ein Stack helfen?

So geht es vielleicht

Da wollen wir hin



Theorie: LR(0)

Arbeitsweise

Im Stack stehen nur Zustandsnummern, am Anfang die Nummer des Startzustandes (+ Bottomzeichen, oft auch $).

- Lesen des obersten Stackelements ergibt Zustand q
- Lesen des nächsten Eingabezeichens ergibt Zeichen a
- Nachschlagen der Reaktion auf (q, a) in der Parse Table
- Durchführung der Reaktion

Mögliche "Actions" ohne Berücksichtigung von Vorschautoken

- Shift: Schiebe logisch das nächste Eingabesymbol auf den Stack (in Wirklichkeit Zustandsnummern)
- Reduce: (Identifiziere ein Handle oben auf dem Stack und ersetze es durch das Nichtterminal der dazugehörigen Produktion.) Das ist gleichbedeutend mit: Entferne so viele Zustände vom Stack wie die rechte Seite der zu reduzierenden Regel Elemente hat, und schreibe den Zustand, der im Goto-Teil für (q, a) steht, auf den Stack.
- Accept: Beende das Parsen erfolgreich

Und was gibt es noch?

Wenn LR(0) nicht reicht

Zunächst: Zu jeder LR(k)-Sprache gibt es eine LR(1)-Grammatik.

Ist eine Grammatik nicht LR(0), müssen nichtdeterminsistische Tabelleneinträge verhindert werden:

- SLR(1)-Parsing ($A \to \beta$ wird nur reduziert, wenn das Vorschautoken in der *FOLLOW*-Menge von A ist.)
- (kanonisches) LR(1)-Parsing (wie LR(0) mit einem Vorschautoken)
- LALR(1)-Parsing (Zusammenfassung aller LR(1)-Zustände, die sich nur in den LOOKAHEAD-Mengen unterscheiden)

Mehrdeutige Grammatiken

Es gibt auch Auswege

Mehrdeutige Grammatiken sind oft leichter zu lesen und kleiner als die Grammatiken, die man erhält, wenn man die Mehrdeutigkeit auflöst, sofern möglich.

Folgendes kann bei Mehrdeutigkeiten helfen:

- Angabe von Vorrangregeln
- Angabe von Assoziativität
- Voreinstellung des Parsergenearators: z. B. Shiften bei Shift-Reduce-Konflikten
- Voreinstellung des Parsergenearators: z. B. Reduzieren nach der Regel, die in der Grammatik zuerst kommt bei Reduce-Reduce-Konflikten

Hierarchie der kontextfreien Sprachen

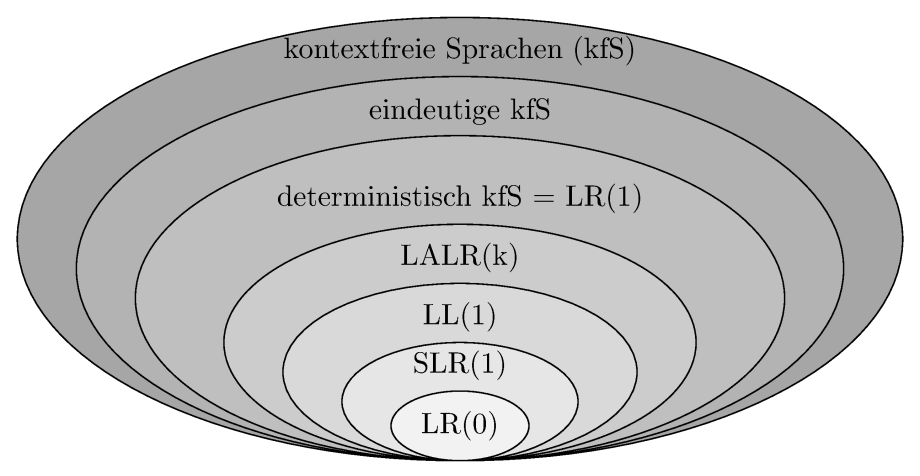


Abbildung 4: Sprachenhierarchie

Wrap-Up

Wrap-Up

- LR-Analyse baut den Ableitungbaum von unten nach oben auf
- es wird ein DFA benutzt zusammen mit einem Stack, der Zustände speichert
- eine Parse-Tabelle steuert über Aktions- und Sprungbefehle das Verhalten des Parsers
- die Tabelle wird mit (dotted) Items und Closures konstruiert
- mit Bottom-Up-Parsing LR(1) kann man alle deterministisch kontextfreien Sprachen parsen
- LR(0)-, SLR- und LALR- Parsing sind vereinfachte Verfahren für Teilmengen der LR-Sprachen

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *e7bbe05 (cleanup: remove 02/lr-parser{1,2}.md - #210, 2025-08-21)*